Chapter 15

1.

```
ubuntu@cs5600:~/ostep-homework/vm-mechanism$ ./relocation.py -s 1 -c
ARG seed 1
ARG address space size 1k
ARG phys mem size 16k

Base-and-Bounds register information:

  Base   : 0x0000363c (decimal 13884)
  Limit  : 290

Virtual Address Trace
  VA  0: 0x0000030e (decimal:  782) --> SEGMENTATION VIOLATION
  VA  1: 0x00000105 (decimal:  261) --> VALID: 0x00003741 (decimal: 14145)
  VA  2: 0x000001fb (decimal:  507) --> SEGMENTATION VIOLATION
  VA  3: 0x000001cc (decimal:  460) --> SEGMENTATION VIOLATION
  VA  4: 0x0000029b (decimal:  667) --> SEGMENTATION VIOLATION
```

For seed 1, only VA 1 is valid. To check the translation that occurs we subtract the base from the physical address to recover the virtual address. In decimal this is 14145 - 13884 = 261.

```
ubuntu@cs5600:~/ostep-homework/vm-mechanism$ ./relocation.py -s 2 -c
ARG seed 2
ARG address space size 1k
ARG phys mem size 16k

Base-and-Bounds register information:

  Base   : 0x00003ca9 (decimal 15529)
  Limit  : 500

Virtual Address Trace
  VA  0: 0x00000039 (decimal:   57) --> VALID: 0x00003ce2 (decimal: 15586)
  VA  1: 0x00000056 (decimal:   86) --> VALID: 0x00003cff (decimal: 15615)
  VA  2: 0x00000357 (decimal:  855) --> SEGMENTATION VIOLATION
  VA  3: 0x000002f1 (decimal:  753) --> SEGMENTATION VIOLATION
  VA  4: 0x000002ad (decimal:  685) --> SEGMENTATION VIOLATION
```

0 -> 15586 - 15529 = 57 1 -> 15615 - 15529 = 86

```
ubuntu@cs5600:~/ostep-homework/vm-mechanism$ ./relocation.py -s 3 -c

ARG seed 3
ARG address space size 1k
```

```
ARG phys mem size 16k

Base-and-Bounds register information:

  Base   : 0x000022d4 (decimal 8916)
  Limit  : 316

Virtual Address Trace
  VA  0: 0x0000017a (decimal:  378) --> SEGMENTATION VIOLATION
  VA  1: 0x0000026a (decimal:  618) --> SEGMENTATION VIOLATION
  VA  2: 0x00000280 (decimal:  640) --> SEGMENTATION VIOLATION
  VA  3: 0x00000043 (decimal:   67) --> VALID: 0x00002317 (decimal: 8983)
  VA  4: 0x0000000d (decimal:   13) --> VALID: 0x000022e1 (decimal: 8929)
```

3 -> 8983 - 8916 = 67 4 -> 8929 - 8916 = 13

2.

```
ubuntu@cs5600:~/ostep-homework/vm-mechanism$ ./relocation.py -s 0 -n 10 -c -l 930

ARG seed 0
ARG address space size 1k
ARG phys mem size 16k

Base-and-Bounds register information:

  Base   : 0x0000360b (decimal 13835)
  Limit  : 930

Virtual Address Trace
  VA  0: 0x00000308 (decimal:  776) --> VALID: 0x00003913 (decimal: 14611)
  VA  1: 0x000001ae (decimal:  430) --> VALID: 0x000037b9 (decimal: 14265)
  VA  2: 0x00000109 (decimal:  265) --> VALID: 0x00003714 (decimal: 14100)
  VA  3: 0x0000020b (decimal:  523) --> VALID: 0x00003816 (decimal: 14358)
  VA  4: 0x0000019e (decimal:  414) --> VALID: 0x000037a9 (decimal: 14249)
  VA  5: 0x00000322 (decimal:  802) --> VALID: 0x0000392d (decimal: 14637)
  VA  6: 0x00000136 (decimal:  310) --> VALID: 0x00003741 (decimal: 14145)
  VA  7: 0x000001e8 (decimal:  488) --> VALID: 0x000037f3 (decimal: 14323)
  VA  8: 0x00000255 (decimal:  597) --> VALID: 0x00003860 (decimal: 14432)
  VA  9: 0x000003a1 (decimal:  929) --> VALID: 0x000039ac (decimal: 14764)
```

I noticed when running it a lot that bigger numbers for the bounds produced better results. When searching for the smallest number that would make all addresses valid, I found that you have to set the bounds register to the largest address generated + 1. So the base doesn't actually matter here; as long as the addresses generated are within the bounds we set, there is no segmentation violation.

3. I saw that when I ran with a very large base I got an error that says:

```
Error: address space does not fit into physical memory with those base/bounds values.
Base + Limit: 10000000   Psize: 16384
```

With that I found that a base of 16284 (you have to factor in the limit of 100) will fit in memory. 16285 will not, as base + limit would then exceed the physical memory size. This checks out because 16k is 16384 bytes.

4. I tried many different values for -a and -p and the rules held. The bounds check is still virtual address < the limit, and base + limit still needs to be no larger than the physical memory size. The translation stays the same even as the sizes scale up.

5. I used `matplotlib` to create the following chart.

```python
import random
import bisect
import matplotlib.pyplot as plt

ASIZE = 1024
NUM_ADDRS = 1000
SEEDS = range(10)

limits = list(range(0, ASIZE + 1))
all_fracs = []

for seed in SEEDS:
    random.seed(seed)
    vas = sorted(int(ASIZE * random.random()) for _ in range(NUM_ADDRS))
    fracs = [bisect.bisect_left(vas, lim) / NUM_ADDRS for lim in limits]
    all_fracs.append(fracs)
    plt.plot(limits, fracs, alpha=0.35, linewidth=1)

avg = [sum(col) / len(col) for col in zip(*all_fracs)]
plt.plot(limits, avg, color="black", linewidth=2, label="average across seeds")
plt.plot(limits, [l / ASIZE for l in limits], "r--", label="limit / address space size")

plt.xlabel("Bounds register value (limit)")
plt.ylabel("Fraction of valid virtual addresses")
plt.title(f"Valid address fraction vs. bounds ({len(SEEDS)} seeds, {NUM_ADDRS} addrs each)")
plt.xlim(0, ASIZE)
plt.ylim(0, 1.05)
plt.grid(True, alpha=0.3)
plt.legend()
plt.show()
```

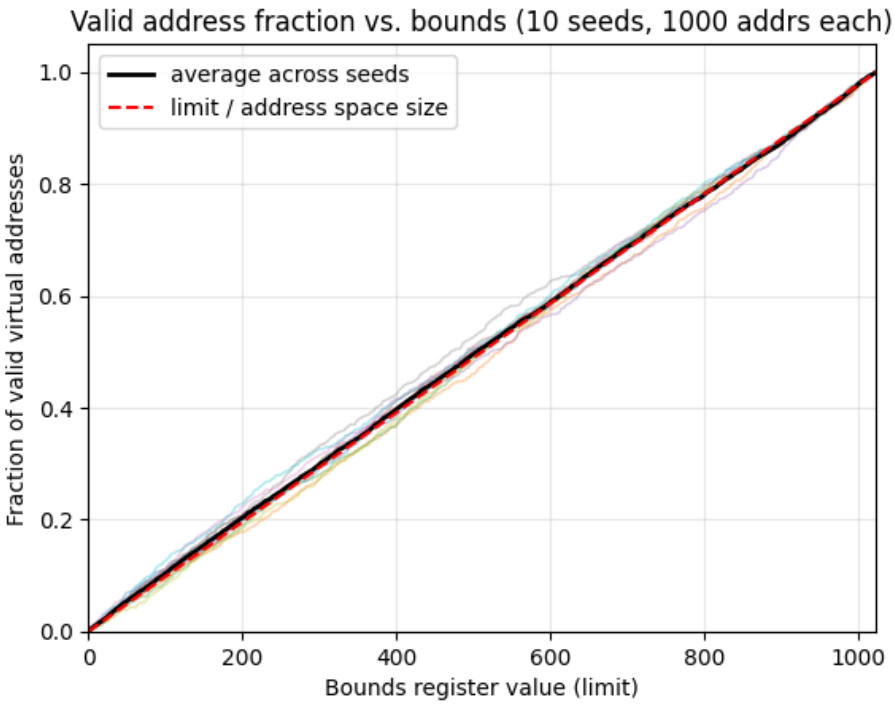


Figure 1: Graph

Chapter 16

1. If we assume 2 segments, as the output states that segments are in 0 or 1, the first segment covers 0 - 63 with the second covering 64 - 127.

```
VA  0: 0x0000006c (decimal:  108) --> PA or segmentation violation?
VA  1: 0x00000061 (decimal:   97) --> PA or segmentation violation?
VA  2: 0x00000035 (decimal:   53) --> PA or segmentation violation?
VA  3: 0x00000021 (decimal:   33) --> PA or segmentation violation?
VA  4: 0x00000041 (decimal:   65) --> PA or segmentation violation?
```

0: The first address (108) would go in segment 1. Since segment 1 grows negatively we need to calculate a negative offset; our physical address will be the base plus that negative offset. The negative offset is 108 - 128 = -20 (we can do this because segment 1 is always the top segment in this simulator, so the offset is really the VA minus the address space size). Since the size of that offset (20) is within the limit of 20, we know that it is not a segmentation violation. Thus the physical address is 512 - 20 = 492.

1: 97 - 128 = -31, |-31| <= 20 -> False, segmentation violation

2: 53 is within segment 0, which grows positively, and it is greater than the limit of 20, so it is a segmentation violation.

3: 33 is also greater than 20, so segmentation violation.

4: 65 - 128 = -63, |-63| <= 20 -> False, segmentation violation

```
VA  0: 0x00000011 (decimal:   17) --> PA or segmentation violation?
VA  1: 0x0000006c (decimal:  108) --> PA or segmentation violation?
VA  2: 0x00000061 (decimal:   97) --> PA or segmentation violation?
VA  3: 0x00000020 (decimal:   32) --> PA or segmentation violation?
VA  4: 0x0000003f (decimal:   63) --> PA or segmentation violation?
```

```
VA  0: 0x0000007a (decimal:  122) --> PA or segmentation violation?
VA  1: 0x00000079 (decimal:  121) --> PA or segmentation violation?
VA  2: 0x00000007 (decimal:    7) --> PA or segmentation violation?
VA  3: 0x0000000a (decimal:   10) --> PA or segmentation violation?
VA  4: 0x0000006a (decimal:  106) --> PA or segmentation violation?
```

2. We know that with a limit of 20 we have 20 valid addresses on either side. Segment 0 is easy because of its positive growth: there are 20 valid addresses here, 0 - 19. Then on the upper side we have 108 - 127. Everything in between is out of bounds, so 20 - 107 will not work.

3. I did the following to get only two valid addresses on each side of the address space. The insight is to set the length to 2 for each and keep base 0 at 0 and base 1 at the physical memory size. This ensures that 0 and 1 will be valid and that 14 & 15 will be valid. Nothing in between can be valid with a length of 2.

```
ubuntu@cs5600:~/ostep-homework/vm-segmentation$ ./segmentation.py -a 16 -p 128 -A 0,1,2,3,4,
```

4. The limit is the most important lever we can pull here to get an exact percentage. Take the example:

fraction_valid = (l0 + l1) / address_space_size

To test this I ran the following:

```
total=0
NUM_RUNS=100
for i in $(seq 1 $NUM_RUNS); do
    count=$(./segmentation.py -a 32 -p 128 --b0 0 --l0 15 --b1 128 --l1 14 -n 100 -s $i -c \
        | grep -o "SEGMENTATION VIOLATION" | wc -l)
    total=$((total + count))
done
echo "scale=2; $total / $NUM_RUNS" | bc
```

The result was an average of 9.34 segmentation violations per run of 100 addresses, which means approximately 90% of addresses were valid. Because we are working with randomness we can't guarantee an exact outcome, but we can get fairly close.

5. If you set `--l0` and `--l1` to 0 then there can be no valid addresses.

Chapter 17

1. When I ran `malloc.py` a few times with a small number of requests I was able to trace each malloc and free. I noticed that over time we were running into what the text describes as external fragmentation. The free list was getting longer and chopped into smaller and smaller pieces.
2. With the worst fit policy the program was finding the largest available chunk to allocate from instead. In this simulation this was mostly okay, but the book says this doesn't perform well at scale.
3. With first fit I noticed that the program was finding the first available chunk of memory. This led to less scanning, as worst and best both need to traverse the whole list. Search time is sped up by doing things this way, but picking the first option is not always the best as we can infer.
4. With `ADDRSORT` I noticed that coalescing was fast and easy because adjacent blocks ended up right next to each other in the list. `SIZESORT+` and `SIZESORT-` had the opposite effect, making coalescing harder but speeding up best & worst fit searches.
5. With `-C` off I noticed that the free list got massive in size. The memory got very fragmented. Eventually bigger requests started to fail even though there was enough total memory, as a result of the fragmentation. When I

tried again with it on, the free list stayed much smaller since coalescing helps us manage the memory slices effectively.

6. As I got closer and closer to 100% with `-P` I saw that almost every malloc request was failing. There was nowhere to put new data, so the free list shrank to nothing. As I got closer to 0% the heap stayed mostly empty. Every request succeeded instantly and the free list was just a couple of large blocks.

7. I noticed an easy way to fragment the memory space was to allocate a sequence of blocks and then free gaps in between them. This fragments the memory space intentionally.

```
python malloc.py -S 100 -A +10,-0,+20,-1,+30
```